Why Semantic Parsing is So Painful (GraphRAG)

17 min read · Jul 6, 2025

Ngoc Follow

If you're not a medium member, please use this link

GraphRAG (Graph Retrieval-Augmented Generation) holds a lot of promise, but making it work in practice is painful — from building the knowledge graph to applying it for tasks like question answering.

In the past, I have tried some framework/platform for constructing knowledge graph from text:

- Microsoft GraphRAG (https://github.com/microsoft/graphrag)
- Neo4j (https://console-preview.neo4j.io)

With Microsoft GraphRAG, at the time I used it, I found the handling of **graph ontology** to be vague and underdeveloped. I also struggled to access or visualize the graph data effectively. When testing with my private dataset, the system often failed to generate correct answers, and I wasn't convinced by the community detection results.

Neo4j provides more flexibility for constructing knowledge graphs, with stronger emphasis on **domain ontology**. It offers better tooling for managing and querying

graph data.

In the end, I believe there is no universal graph-building framework that works perfectly across all use cases or domains. To get the most out of GraphRAG, it's essential to understand the core concepts behind GraphRAG and their limitations. Only then can we design custom solutions tailored to our specific problems, ultimately enhancing the performance of GraphRAG-based systems.

In this post, I will focus on one of the key retrieval approaches in GraphRAG: **Semantic Parsing**. I'll walk you through why semantic parsing is challenging in practice and share how we can overcome those difficulties to make GraphRAG more effective.

An overview of Semantic Parsing

Semantic Parsing is a crucial component of Natural Language Understanding (NLU) that focuses on converting natural language text into machine-readable, formal meaning representations — often in the form of logical structures or graphs. In simple terms, semantic parsing is the process of transducing natural language utterances into structured, formal outputs that a machine can understand and execute.

Semantic parsing has been a core machine learning problem for decades. Historically, traditional approaches relied heavily on rule-based systems, which required significant manual engineering and struggled to generalize across domains.

With the rise of deep learning, the performance of semantic parsing has improved dramatically. One of the most common modern approaches is to formulate it as a **sequence-to-sequence** task, leveraging the same architectures that power machine translation models.

> *You can explore benchmark datasets and models for semantic parsing on Papers With Code.*

Apply Semantic Parsing in GraphRAG

As data is represented in structure format — knowledge graph, **Semantic Parsing** can play a vital role during the retrieval phase. More specifically, user queries expressed in natural language can be parsed into a formal graph query language, such as **Cypher**, which allows machines to execute precise graph queries and retrieve relevant information to support answer generation.

By leveraging semantic parsing in this way, we can bridge the gap between unstructured user inputs and the structured nature of knowledge graphs, enabling more accurate and interpretable retrieval in GraphRAG systems.

For example:

List the side effects of Aspirin

Parsed Cypher query:

```
MATCH (d:Drug {name: "Aspirin"})-[:HAS_SIDE_EFFECT]->(se:SideEffect)
RETURN se.name
```

Now let's deep dive into Semantic Parsing in GraphRAG pipeline.

Experiment

Knowledge Graph Data

For this work, I use the **medical knowledge graph dataset** that I previously constructed, as explained in my earlier post. Specifically, my knowledge graph combines two layers of structure:

- **Document Hierarchy**: representing the structure within documents (e.g., sections, paragraphs).
- **Domain Knowledge**: representing entities and relationships specific to the medical field (e.g., diseases, drugs, symptoms, and their connections).

I designed my graph ontology by using structured data objects to model different node types. There are three core node types in my graph:

1. **Doc Node**: Inherits from `BaseDoc`, representing an entire document as a single node in the graph.

```
class BaseDoc(OntologyEntity, Generic[UnitT]):
    id: Optional[str] = Field(None, description="The id of the document.")
    title: Optional[str] = Field(None, description="The title of the document."
    units: List[UnitT] = Field(..., description="The units of the document.")

    @classmethod
    def node_label(cls) -> str:
        return "Doc"

    def model_post_init(self, context: Any, /) -> None:
        if self.id is None:
            self.id = f"Doc_{str(self.next_id()).zfill(4)}"
    ...
```

2. **Unit/Section Node**: Inherits from `BaseDocUnit`, representing smaller units within the document hierarchy, such as sections, paragraphs, or chunks. These nodes capture the internal structure of documents and preserve context at different levels of granularity.

```
class BaseDocUnit(OntologyEntity):
    id: Optional[str] = Field(None, description="The id of the document.")

    title: Optional[str] = Field(None, description="The title of the paragraph.
    text: str = Field(..., description="The text of the paragraph. Extracted fr

    mentions: Optional[List[BaseMention]] = Field([], description="Default is e
    relationships: Optional[List[BaseDomainRelation]] = Field(
        [],
        description="Default is empty",
    )

    def model_post_init(self, context: Any, /) -> None:
        if self.id is None:
            self.id = f"Unit_{str(self.next_id()).zfill(4)}"
    ...
```

3. Mention Node: Inherits from `BaseMention`, representing specific entities or concepts mentioned within the text. Mentions can refer to diseases, drugs, symptoms, or other medically relevant terms extracted from the content.

```
class BaseMention(OntologyEntity):
    id: Optional[str] = Field(None, description="The entity identifier.")

    entity_type: str
    text: str = Field(..., description="Mention string appears in the passage")
    summary: Optional[str] = Field(None, description="Summary description of th

    def model_post_init(self, context: Any, /) -> None:
        self.summary = self.text

    @classmethod
    def node_label(cls) -> str:
        return "Mention"

    def __str__(self):
        return f"<{self.entity_type}>{self.text}</{self.entity_type}>"
```

All of these base models — `BaseDoc`, `BaseDocUnit`, and `BaseMention` — inherit from a common class called **`OntologyEntity`**, which represents a **node** in the knowledge graph.

`OntologyEntity` defines the shared structure and properties for all nodes, such as unique IDs, node embedding, making the graph consistent and easy to extend across different domains or tasks.

This design provides a flexible foundation to build complex, domain-specific knowledge graphs while keeping the underlying structure maintainable and scalable.

```
class OntologyEntity(BaseModel, ABC):
    _id_counter: ClassVar[Iterator[int]] = count(1)
    _registry: ClassVar[Dict] = {}
    _relationship_map: ClassVar[Dict[str, Dict[str, str]]] = {
        "Unit": {
            "Mention": "HAS_MENTION",
            "Unit": "NEXT_UNIT",
```

```
        },
        "Doc": {
            "Unit": "HAS_UNIT",
        }
    }

    id: str
    summary: str = Field(..., description="Summary description of the instance.
    embedding: Optional[List[float]] = Field(None, description="The embedding v

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        if not getattr(cls, "__abstract__", False):
            OntologyEntity._registry[cls.node_label()] = cls.model_json_schema(

    @classmethod
    def update_registry(cls, model_class, node_label=None):
        """Manually update the registry with a model class."""
        label = node_label or model_class.node_label()
        cls._registry[label] = model_class.model_json_schema()


    @classmethod
    def next_id(cls) -> int:
        return next(cls._id_counter)

    @classmethod
    @abstractmethod
    def node_label(self) -> str:
        """Returns the node label for this entity."""
        pass

    @classmethod
    def get_registered_entities(cls) -> List[Dict[str, str]]:
        """
        Returns all registered entity labels and their docstrings for serializa
        """
        return [
            {
                "label": node_label,
                "model": subclass
            }
            for node_label, subclass in cls._registry.items()
        ]

    @classmethod
    def get_available_relationships(cls) -> List[Dict[str, str]]:
        """Returns relationships for serialization."""
        result = []
        for subj, objs in cls._relationship_map.items():
            for obj, name in objs.items():
                result.append({
                    "subject": subj,
```

```
                    "object": obj,
                    "relationship": name
                })
        return result


    @classmethod
    def dump_ontology_schema(cls) -> Dict[str, Any]:
        """Return a JSON-serializable representation of the ontology."""
        return {
            "entities": cls.get_registered_entities(),
            "relationships": cls.get_available_relationships()
        }

    @classmethod
    def save_ontology_schema(cls, filepath: str):
        """Save schema as JSON file."""
        with open(filepath, "w") as f:
            import json
            json.dump(cls.dump_ontology_schema(), f, indent=4)

    @classmethod
    def infer_relationship(cls, subject_label: str, object_label: str) -> str:
        """Centralized ontology relationship inference."""
        return cls._relationship_map.get(subject_label, {}).get(object_label)

    def get_relationships(self) -> list:
        """By default, entities have no outgoing relationships."""
        return []

    def node_repr(self) -> str:

        string_repr = ""

        for attr_name, attr_value in vars(self).items():
            if attr_name != "id" and isinstance(attr_value, str):
                string_repr += f"{attr_name}: {attr_value}\n"

        return string_repr
```

In addition to nodes, **relationships** in the knowledge graph are also modeled as structured data objects

```
class RelationshipInstance(BaseModel):
    type_ : str
    subject_id: str
```

```
    object_id: str
    properties: dict = {}
```

Semantic Parsing Model

In this work, I use a Large Language Model (LLM), specifically **GPT-4o**, to perform semantic parsing. One advantage of GPT-4o is that it has been exposed to structured query languages like **Cypher** during its training, enabling it to understand and generate graph queries from natural language.

Tech stack

In this work, I use an agent framework called **PydanticAI** to build the LLM-based application. PydanticAI helps structure interactions between the LLM, tools, and external systems by leveraging `pydantic` models for strong typing and data validation.

That said, it's completely fine if you're not familiar with this framework — the core idea is independent of any specific tool. You can implement the same logic and structure on your own.

From now I'll present **step by step the issues I have encountered** and how I solved or partially solved it.

Issue 1: LLMs don't align with your Domain Ontology

When I first applied semantic parsing as a retrieval approach in my GraphRAG pipeline, I naively asked the LLM to directly convert natural language queries into Cypher queries.

However, I quickly realized that without any domain-specific guidance or context, the LLM struggled to correctly map user queries to the precise graph ontology and relationships in my medical knowledge graph.

```
from pydantic_ai.models.openai import OpenAIModelSettings
from pydantic_ai import Agent, RunContext
from src._base import instruct_model

def create_semantic_parser_agent() -> Agent:
    agent = Agent(
        name="semantic_parser_agent",
        model=instruct_model,
        output_type=str,
        retries=5,
        model_settings=OpenAIModelSettings(
            temperature=0.1,
        ),
        system_prompt="""
        "You are a helpful assistant that translates natural language into Cyph
        "You should only return Cypher queries, not other text.\n"
        """
    )

    return agent
```

```
import nest_asyncio
nest_asyncio.apply()

res = agent.run_sync("List the side effects of Aspirin")
```

Output:

```
MATCH (d:Drug {name: "Aspirin"})-[:HAS_SIDE_EFFECT]->(s:SideEffect)
RETURN s.name AS SideEffect
```

The output doesn't align my ontology schema, I don't have a **`SideEffect` node** in the graph.

Solution: Teach the LLM Your Ontology

The solution is simple but critical: you need to **inject your ontology schema into the LLM's context** and explicitly guide it to parse queries in alignment with that schema.

In my case, the `OntologyEntity` class includes a method called `save_ontology_schema`, which is responsible for exporting the graph schema — including both the **document structure** and the **domain ontology** — into a JSON file during the knowledge graph construction phase.

Later, I load this JSON schema and include it in the **system prompt** when interacting with the LLM. By providing this schema, the LLM can parse natural language queries into Cypher in a way that is consistent with the actual graph, avoiding hallucinations or incorrect references to non-existent entities or relationships.

Other framework like Neo4j use RDF format, but I found json format can work well too:


```
{
    "entities": [
        {
            "label": "Mention",
            "model": {
                "properties": {
                    "id": {
                        "title": "Id",
                        "type": "string"
                    },
                    "summary": {
                        "description": "Summary description of the instance.",
                        "title": "Summary",
                        "type": "string"
                    },
                    "embedding": {
                        "anyOf": [
                            {
                                "items": {
                                    "type": "number"
                                },
                                "type": "array"
                            },
                            {
                                "type": "null"
                            }
                        ],
                        "default": null,
                        "description": "The embedding vector.",
                        "title": "Embedding"
                    }
                },
                "required": [
```

```
                    "id",
                    "summary"
                ],
                "title": "OntologyEntity",
                "type": "object"
            }
        },

        ...

    ],
    "relationships": [
        {
            "subject": "Unit",
            "object": "Mention",
            "relationship": "HAS_MENTION"
        },
        {
            "subject": "Unit",
            "object": "Unit",
            "relationship": "NEXT_UNIT"
        },
        {
            "subject": "Doc",
            "object": "Unit",
            "relationship": "HAS_UNIT"
        }
    ]
}
```

```
{
    "domain_name": "medical",
    "entity_types": [
        {
            "name": "DRUG",
            "description": "A substance used to treat or prevent diseases or sy
        },
        {
            "name": "DISEASE",
            "description": "A disorder or abnormal condition affecting the body
        },
        {
            "name": "SYMPTOM",
            "description": "A physical or mental feature that indicates a condi
        },
        {
            "name": "VIRUS",
            "description": "A microscopic infectious agent that can cause disea
        }
```

```
        ],
        "relationship_types": [
            {
                "name": "TREATS",
                "description": "Indicates that a drug is used to treat a disease or
                "allowed_type_pairs": [
                    {
                        "subject_type": "DRUG",
                        "object_type": "DISEASE"
                    },
                    {
                        "subject_type": "DRUG",
                        "object_type": "SYMPTOM"
                    }
                ]
            },
            {
                "name": "CAUSES",
                "description": "Indicates that a virus or other agent causes a dise
                "allowed_type_pairs": [
                    {
                        "subject_type": "VIRUS",
                        "object_type": "DISEASE"
                    },
                    {
                        "subject_type": "DRUG",
                        "object_type": "SYMPTOM"
                    }
                ]
            },
            {
                "name": "HAS_SIDE_EFFECT",
                "description": "Indicates that a drug has a side effect which is a
                "allowed_type_pairs": [
                    {
                        "subject_type": "DRUG",
                        "object_type": "SYMPTOM"
                    }
                ]
            },
            {
                "name": "HAS_SYMPTOM",
                "description": "Indicates that a disease has associated symptoms.",
                "allowed_type_pairs": [
                    {
                        "subject_type": "DISEASE",
                        "object_type": "SYMPTOM"
                    }
                ]
            }
        ]
    }
```

```
from pydantic_ai.models.openai import OpenAIModelSettings
from pydantic_ai import Agent, RunContext
from src._base import instruct_model
from src.graphrag.retrieval.dependency import Dep

def create_semantic_parser_agent() -> Agent:
    agent = Agent(
        name="semantic_parser_agent",
        model=instruct_model,
        output_type=str,
        retries=5,
        model_settings=OpenAIModelSettings(
            temperature=0.1,
        ),
        system_prompt="""
        "You are a helpful assistant that translates natural language into Cyph
            "You should only return Cypher queries, not other text.\n"
        """
    )
    @agent.system_prompt(dynamic=True)
    def create_system_prompt(ctx: RunContext[Dep]) -> str:
        import json

        domain_ontology = json.load(open(ctx.deps.domain_ontology_path))
        doc_structure = json.load(open(ctx.deps.doc_structure_path))

        sys_prompt = (
            "You are a helpful assistant that translates natural language into
            "You should only return Cypher queries, not other text.\n"
            "Important: Follow the ontology and document structure to generate
            "\n"
            "Domain Ontology:\n"
            f"{str(domain_ontology)}\n"
            "Document Structure:\n"
            f"{str(doc_structure)}\n"
        )
        return sys_prompt

    return agent
```

```
import nest_asyncio

nest_asyncio.apply()
```

```
from neo4j import GraphDatabase

res = agent.run_sync("List the side effects of Aspirin",
                     deps=Dep(
                         domain_ontology_path="/home/ju/PycharmProjects/automat
                         doc_structure_path="/home/ju/PycharmProjects/automated
                         driver=GraphDatabase.driver("bolt://localhost:7687", a
                     ))
```

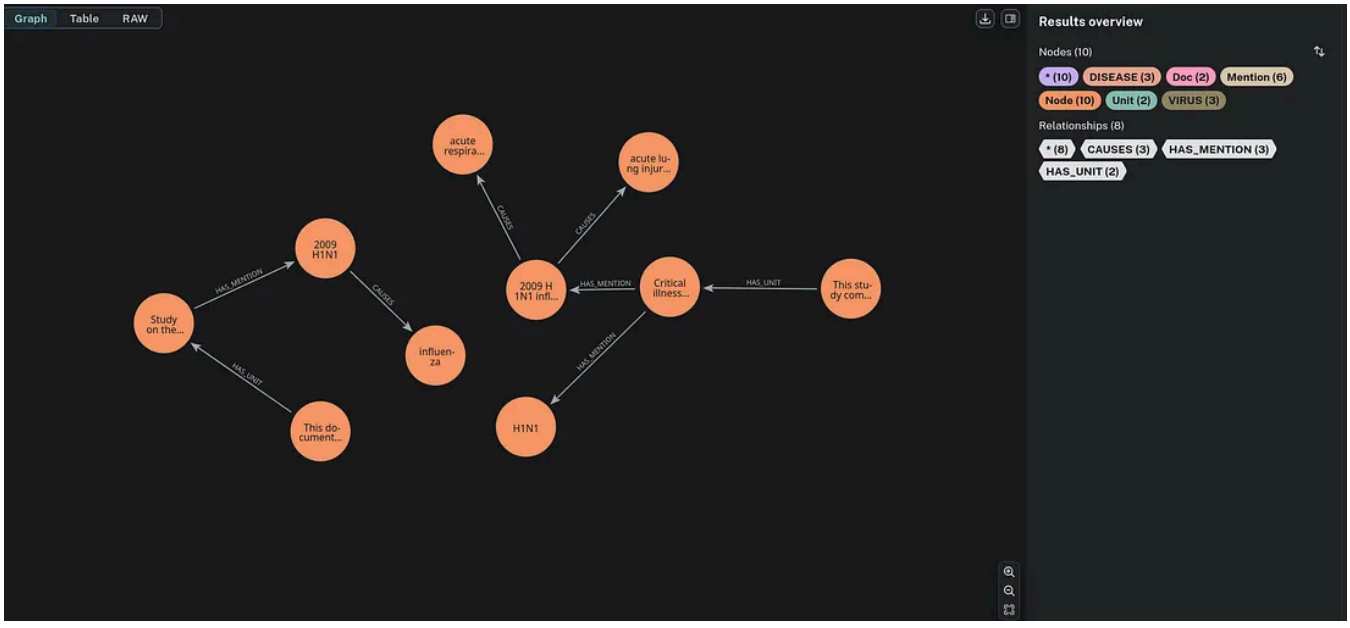
Output:

```
MATCH (d:DRUG {name: "Aspirin"})-[:HAS_SIDE_EFFECT]->(s:SYMPTOM)
RETURN s.name AS SideEffect
```

✅ Now it's perfectly aligned with the Domain Ontology.

Issue 2: Natural language is inherently ambiguous.

Let's take a look at another example on my GraphDB:



There are two papers in my database that mention the **H1N1 virus**. However, when I ask the LLM: *"Which papers study swine flu?"*

This is the Cypher query generated by the LLM:

```
MATCH (d:Doc)-[:HAS_UNIT]->(u:Unit)-[:HAS_MENTION]->(m:Mention)
WHERE toLower(m.text) CONTAINS 'swine flu'
RETURN DISTINCT d.id AS PaperId, d.title AS PaperTitle
```

Technically, the Cypher query is syntactically correct, but semantically misaligned with my Knowledge Graph. The LLM simply extracts the mention “swine flu” from the user’s query and injects it into the Cypher, without considering how the concept is represented in the graph.

In my data, the term “H1N1” is used to represent **swine flu**, so querying for “swine flu” based on raw mention text may not return the correct or complete results.

Solution: Enhance Cypher Query with Entity Linking

Entity Linking is the process of mapping mentions in natural language to the corresponding entities in a Knowledge Graph. It is a core task in both **Information Extraction (IE)** and **Information Retrieval (IR)**. For reference:
Papers with Code — Entity Linking

> “Assigning a unique identity to entities (such as famous individuals, locations, or companies) mentioned in text.”

While advanced approaches use Deep Neural Networks for entity linking, in this work I adopt a lightweight solution based on **similarity search**, which is sufficient for aligning query terms with existing KG entities.

To apply **Entity Linking** in the Semantic Parsing process, I have experimented with several approaches.

Approach 1: 3-Step Pipeline

- **Step 1: Mention Extraction:** Identify and extract all mentions from the user’s query.
- **Step 2: Entity Linking:** For each mention, generate the top-k candidate entities from the Knowledge Graph based on similarity or other matching techniques.
- **Step 3: Context Injection for Semantic Parsing —** Inject the entity candidates back into the prompt or context to guide the LLM in generating the correct Cypher query.

However, through experimentation, I realized that injecting entity candidates directly into the context can disrupt the natural reasoning process of the LLM. The model tends to overfit to the extracted mentions and forcefully use them all in the generated Cypher query, which is often incorrect. For example, when I query: *"Which virus causes flu?"*

There are two extracted mentions: **virus** and **flu**. But logically, only **flu** should be used as a filtering condition in the Cypher, because we are looking for viruses that cause flu — not filtering viruses by "virus" (which is redundant) and "flu" simultaneously. This highlights a key challenge: naively injecting all entity candidates can mislead the model, leading to incorrect or irrelevant Cypher queries.

Approach 2: Template-Based Query Generation (The Method Used in This Work)

In this work, I adopt a more effective approach that preserves the natural flow of semantic parsing.

- **Step 1: Generate a Template Query with Placeholder Mentions —** Instead of injecting entity candidates directly into the context, I first guide the LLM to generate a **template Cypher query** with placeholders for mentions. This step allows the LLM to focus purely on **parsing the structure and intent of the query**, without being biased by the entity linking process.The output of this step is a `QueryPlaceholder` object, which contains: The **query structure** and **Mention placeholders** that need to be resolved later

```
class EntityMention(BaseModel):
    text: str = Field(..., description="The mention extracted from the query")
    placeholder: str = Field(..., description="The corresponding placeholder ir

class QueryPlaceholder(BaseModel):
    query: str = Field(..., description="The Cypher query with placeholder. E.g
    mentions: List[EntityMention] = Field(..., description="Maps placeholder ir
```

- **Step 2: Entity Linking —** Once I have the template query with placeholders, the next step is to resolve those placeholders through **Entity Linking**. For each extracted mention in the query, I perform an entity linking task to identify possible matching entities in the Knowledge Graph. The process returns **top-k**

entity candidates for each mention, based on similarity search or other lightweight techniques.

```
def enhance_query(driver, query: QueryPlaceholder) -> Dict:
    """Enhance the query with entity linking.
    Output will be a dictionary with mention as key and a list of entity candic
    """

    entity_candidates = defaultdict(list)
    for mention in list(query.mentions):
        mention_embedding = embedding(mention.text)

        with driver.session() as session:
            search_query = f"""
                            CALL db.index.vector.queryNodes('node_embedding
                            RETURN node, score
                            """

            search_result = session.run(search_query, embedding=mention_embeddi

            for record in search_result:
                props = {}
                for key, value in record["node"].items():
                    props[key] = value
                entity_candidates[mention.text].append(
                    props["id"]
                )

    return entity_candidates
```

- **Step 3: Re-generate Cypher Query with Candidate Entity IDs —** In this step, I replace the placeholder mentions with the corresponding **entity IDs** obtained from the Entity Linking process, to produce the final executable Cypher query. Technically, it might be possible to implement a logic function to simply replace `.text` with `.id` in the generated Cypher. However, this is not always reliable, because there are many ways to structure a Cypher query that produce the same result, and hardcoded replacements can easily break or miss certain patterns. Instead of relying on rigid replacements, I take a more flexible approach: I use **another LLM agent** to generate the final Cypher query, using both: The **template query with placeholders** The **resolved entity IDs .** This allows the model to

regenerate the query naturally, grounded in the correct entity identifiers from the Knowledge Graph, while preserving flexibility in query structure.

```
@dataclass
class SemanticParserNode(BaseNode[None, Dep, End]):
    user_query: str
    semantic_parser_agent: Agent = create_semantic_parser_agent()
    query_enhancer_agent: Agent = create_query_enhancer_agent()

    ontology_models = {}

    def build_query_enhancement_content(self, cypher_template: QueryPlaceholder
        return f"""USER QUERY: {self.user_query}\n\n
                CYPHER TEMPLATE QUERY: {str(cypher_template)}\n\n
                ENTITY CANDIDATES: {str(entity_candidates)}
        """

    async def run(self,
                  ctx:GraphRunContext[None, Dep]) -> ResponseGeneratorNode:

        # Load ontology entity models
        import json

        doc_structures = json.load(open(ctx.deps.doc_structure_path))
        base_model_mapping = {
            "Doc":BaseDoc,
            "Unit":BaseDocUnit,
            "Mention":BaseMention,
        }
        for entity in doc_structures['entities']:
            entity_label = entity['label']
            entity_schema = entity['model']

            self.ontology_models[entity_label] = json_schema_to_base_model(enti

        # Step 1
        response = await self.semantic_parser_agent.run(
            user_prompt=self.user_query,
            deps=ctx.deps,
        )

        cypher_query: QueryPlaceholder = response.output
        print(cypher_query.query)

        # Step 2
        entity_candidates = enhance_query(ctx.deps.driver, cypher_query)

        # Step 3
        response = await self.query_enhancer_agent.run(
            user_prompt=self.build_query_enhancement_content(cypher_query, enti
```

```
            deps=ctx.deps,
        )
        print(response.output)
```

✅ **Output:**

```
11:05:42.934   run node SemanticParserNode
11:05:42.943     semantic_parser_agent run
11:05:42.948       chat gpt-4.1-mini
MATCH p=(doc)-[:HAS_UNIT]->(unit)-[:HAS_MENTION]->(mention)
WHERE mention.text = 'PLACE_HOLDER_1'
RETURN p
11:05:46.783     query_enhancer_agent run

{'summary': 'H1N1', 'entity_type': 'VIRUS', 'text': 'H1N1', 'id': '0068'}
0.8063950538635254
{'summary': '2009 H1N1 influenza', 'entity_type': 'VIRUS', 'text': '2009 H1N1 i
0.7910223007202148
{'summary': '2009 H1N1', 'entity_type': 'VIRUS', 'id': '0070', 'text': '2009 H1
0.7866525650024414

11:05:46.789       chat gpt-4.1-mini
11:05:48.013       chat gpt-4.1-mini
MATCH p=(doc)-[:HAS_UNIT]->(unit)-[:HAS_MENTION]->(mention)
WHERE mention.id IN ['0068', '0062', '0070']
RETURN p

Two papers in the provided context research about swine flu (2009 H1N1 influenz

1. "Relative cost and outcomes in the intensive care unit of acute lung injury

2. "Triple Combination of Amantadine, Ribavirin, and Oseltamivir Is Highly Acti

Both papers focus on aspects related to the 2009 H1N1 influenza (swine flu).
```

Optional:

Looking at the output above, suppose we already know that our query should only focus on the following mentions by their IDs:

```
WHERE mention.id IN ['0068', '0062', '0070']
```

This insight allows us to further enhance the **Semantic Parsing** process.

Instead of giving the LLM complete freedom to generate arbitrary Cypher patterns , which can lead to incorrect, irrelevant, or overly complex queries, we can constrain the model's reasoning by providing the **available path patterns** upfront.

Specifically, we can:

- Query the Knowledge Graph for **1-hop**, **2-hop**, or **3-hop** paths surrounding those `node_id`s
- Extract the valid, domain-consistent connection patterns
- Feed these patterns as context to the LLM during Cypher generation

This approach provides two major benefits:

1. It reduces the risk of generating invalid or nonsensical graph patterns.
2. It allows the LLM to reason within the boundaries of the actual graph structure, improving both accuracy and alignment with the Knowledge Graph

Optional: Adding a Retry Loop with Post-Validation

To improve the reliability of Semantic Parsing in GraphRAG, you can implement a **post-validation step** for the generated Cypher queries. This allows the system to automatically detect invalid queries and force the LLM to regenerate them.

Here's a code snippet showing how to integrate output validation into your Agent:

```
    @agent.output_validator
    async def validate_output(ctx: RunContext[Dep], output: QueryPlaceholder) -
        """Validate the Cypher query. Force the model to re-generate if there i
        try:
            with ctx.deps.driver.session() as session:
                session.run(output.query)
        except Exception as e:
            raise ModelRetry(f"Invalid Cypher query: {e}")
        return output

    return agent
```

If validation fails (e.g., due to syntax or logical issues), the `ModelRetry` exception prompts the LLM to regenerate the Cypher query.

Note:
Each retry includes the attempt history and the error message when sent to the Agent. This helps the LLM avoid making the same mistake, but it also increases token usage and could raise costs if multiple retries are triggered.

Issue 3: Handling query results to provide context.

This is another important insight I've encountered during development:

1. **How do we want the context to look when passing information back to the LLM?**

In some cases, the LLM successfully generates a Cypher query that retrieves the correct object needed to answer the user's question. For example, take the following Cypher:

```
MATCH (d:DRUG {name: "Aspirin"})-[:HAS_SIDE_EFFECT]->(s:SYMPTOM)
RETURN s.name AS SideEffect
```

You might have the result might look like:: "headache" or `{ "SideEffect": "headache"` . At first glance, this seems correct. But in reality, issues may still exist from previous steps, such as **imperfect Entity Linking**, which means the query could return misleading or unrelated results.

Idea: Return Graph Paths (Triplets) Instead of Raw Nodes

Instead of returning only the final node properties (e.g., `"headache"`), it may be better to guide the LLM to return **graph paths** or **triplets.** *E.g: aspirin - has_side_effect -> headache.* This provides:

- **Transparent context** for the LLM to reason over the relationship. Give it a chance to refuse to answer if the context is irrelevant to the question.
- A way to double-check if the result makes sense, even if earlier steps like entity linking were imperfect.

For example you can see my previous example (#2), the query return path instead of just a node.

2. Process query result is complicated

Another challenge I encountered lies in handling the query results returned from Cypher execution.

In my system, a **node** is not just a simple text label, it's modeled as a structured **data object** with multiple properties (Though another option is flatten everything, change properties into relationships). This provides flexibility and aligns with the domain model, but it also adds complexity when processing results.

Moreover, due to the wide variety of ways to write Cypher queries that achieve the same logical outcome, it's difficult to design a **generic processing logic** that works seamlessly for all possible query structures.

My Design Choice: Data Object Modeling

As I model each node as a **data object**, with clear mappings between graph nodes and domain-level objects. This approach has two benefits:

- It simplifies the process of converting query results back into usable, structured objects
- It makes the entire system more **transparent**, maintainable, and aligned with application-level logic

By standardizing how nodes are represented, I can reduce the unpredictability of query results and make post-processing more manageable, even when dealing with diverse Cypher query patterns. You can see my `OntologyEntity` has a method `node_repr()` to represent nodes in context.

The final Semantic Parsing logic:

```
@dataclass
class SemanticParserNode(BaseNode[None, Dep, End]):
    user_query: str
    semantic_parser_agent: Agent = create_semantic_parser_agent()
    query_enhancer_agent: Agent = create_query_enhancer_agent()
```

```
    ontology_models = {}

    def build_query_enhancement_content(self, cypher_template: QueryPlaceholder
        return f"""USER QUERY: {self.user_query}\n\n
                CYPHER TEMPLATE QUERY: {str(cypher_template)}\n\n
                ENTITY CANDIDATES: {str(entity_candidates)}
        """

    async def run(self,
                  ctx:GraphRunContext[None, Dep]) -> ResponseGeneratorNode:

        # Load ontology entity models
        import json

        doc_structures = json.load(open(ctx.deps.doc_structure_path))
        base_model_mapping = {
            "Doc":BaseDoc,
            "Unit":BaseDocUnit,
            "Mention":BaseMention,
        }
        for entity in doc_structures['entities']:
            entity_label = entity['label']
            entity_schema = entity['model']

            self.ontology_models[entity_label] = json_schema_to_base_model(enti
                                                                           base

        response = await self.semantic_parser_agent.run(
            user_prompt=self.user_query,
            deps=ctx.deps,
        )

        cypher_query: QueryPlaceholder = response.output
        print(cypher_query.query)
        entity_candidates = enhance_query(ctx.deps.driver, cypher_query)

        response = await self.query_enhancer_agent.run(
            user_prompt=self.build_query_enhancement_content(cypher_query, enti
            deps=ctx.deps,
        )
        print(response.output)

        def get_node_info(node) -> OntologyEntity | None:
            props = {}
            for key, value in node.items():
                props[key]=value
            props.pop("embedding", None)

            node_labels = node.labels

            for model_name, ontology_model in self.ontology_models.items():
                if model_name in node_labels:
                    for field_name, field_info in ontology_model.model_fields.i
```

```
                    if field_name not in props and field_info.is_required()
                        # Get the field type annotation
                        field_type = field_info.annotation

                        # Set the default value based on type
                        if field_type == str:
                            props[field_name] = ""
                        elif field_type == list or str(field_type).startswi
                                field_type).startswith('list'):
                            props[field_name] = []
                        elif field_type == dict or str(field_type).startswi
                                field_type).startswith('dict'):
                            props[field_name] = {}
                        else:
                            # For other types, try None first
                            props[field_name] = None
                return ontology_model(**props)
        return None

    triplets = []
    with ctx.deps.driver.session() as session:
        query_result = session.run(response.output)
        for record in query_result:
            path = record['p']  # Get the path object

            # Extract relationships from path
            relationships = path.relationships

            # Form triplets from relationships
            for rel in relationships:
                subject = rel.start_node  # Get start node from relationshi
                predicate = rel.type  # Get the relationship type
                object_ = rel.end_node  # Get end node from the relationshi

                subject_node: OntologyEntity = get_node_info(subject)
                object_node: OntologyEntity = get_node_info(object_)

                triplets.append([
                    subject_node.node_repr(),
                    predicate,
                    object_node.node_repr()
                ])

    return ResponseGeneratorNode(user_query=self.user_query, retrieved_cont
```

Conclusion

Building a robust GraphRAG system requires more than simply combining LLMs with a Knowledge Graph, it demands precise handling of **Semantic Parsing**, entity

grounding, and query reliability.

In this post, I shared practical insights on enhancing Semantic Parsing in GraphRAG, especially how to align LLM-generated queries with your domain ontology and Knowledge Graph structure.

I hope you found these ideas useful. Stay tuned — there are many interesting topics coming :)

Llm Agent | Neo4j | Knowledge Graph | NLP | Computer Science

Follow

Written by Ngoc

247 followers · 32 following

AI Engineer with a passion for NLP, agents, knowledge graphs, and building intelligent systems. Always learning, always exploring. Feel free to contact me!

Responses (8)

Deepak Gupta

What are your thoughts?

Govind Diwakar
Jul 8

Very interesting

9 1 reply Reply

Raj Krishnan
Jul 7

Very interesting article. Thank you.

13 1 reply Reply

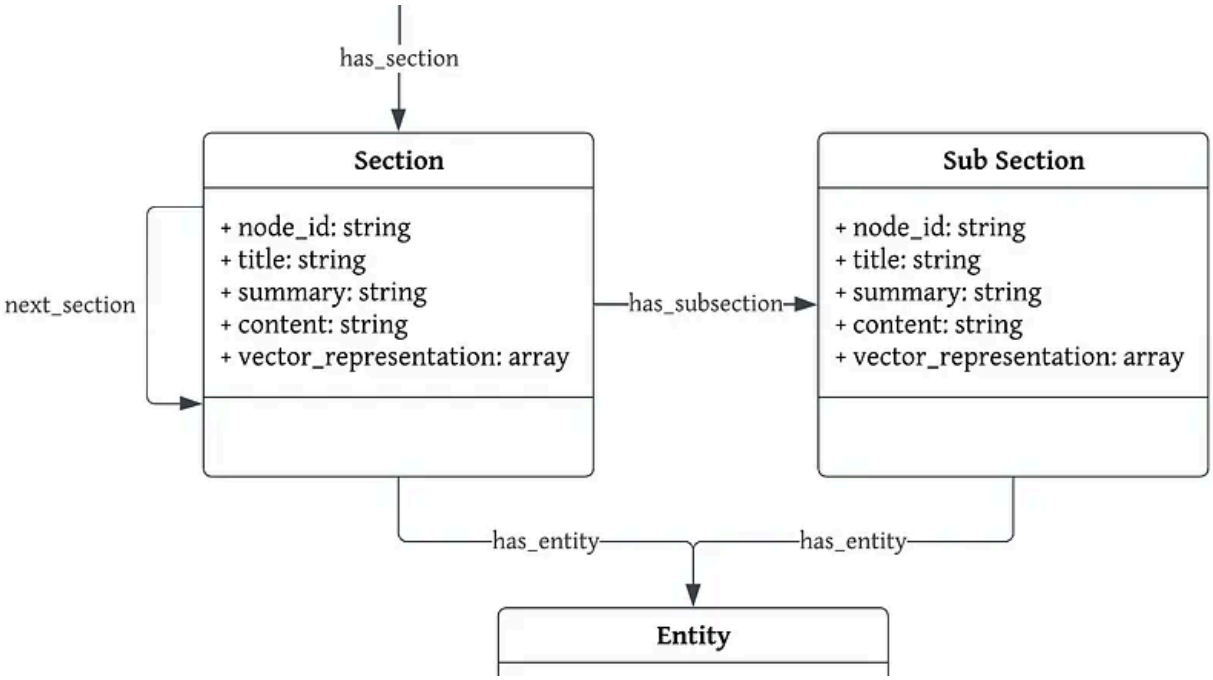
Michael Iantosca
Jul 13

There are data -based graphs and document-based graphs. For the latter I prefer to use RDF and OWL as they better support linked data and inference. Also, when dealing with documents as source, performance is better with containerized larger... more

6 1 reply Reply

See all responses

More from Ngoc



Ngoc

Build an Agent for Document-Based Knowledge Graphs Construction (GraphRAG)

This post introduces a modular, agent-based approach to building document-centric knowledge graphs using PydanticAI. By delegating tasks...

May 27 187 1

Ngoc

From Human-in-The-loop Knowledge graph Construction to Agentic GraphRAG.

A practical guide to building robust, purpose-driven knowledge graphs in the medical domain using human-in-the-loop agentic workflows...

Jun 13 91

Ngoc

How I Run Ollama on Google Colab to Learn AI Agents with Pydantic AI

Run powerful local LLMs in Google Colab using Ollama, and build structured AI agents with Pydantic AI...

May 24 12

Ngoc

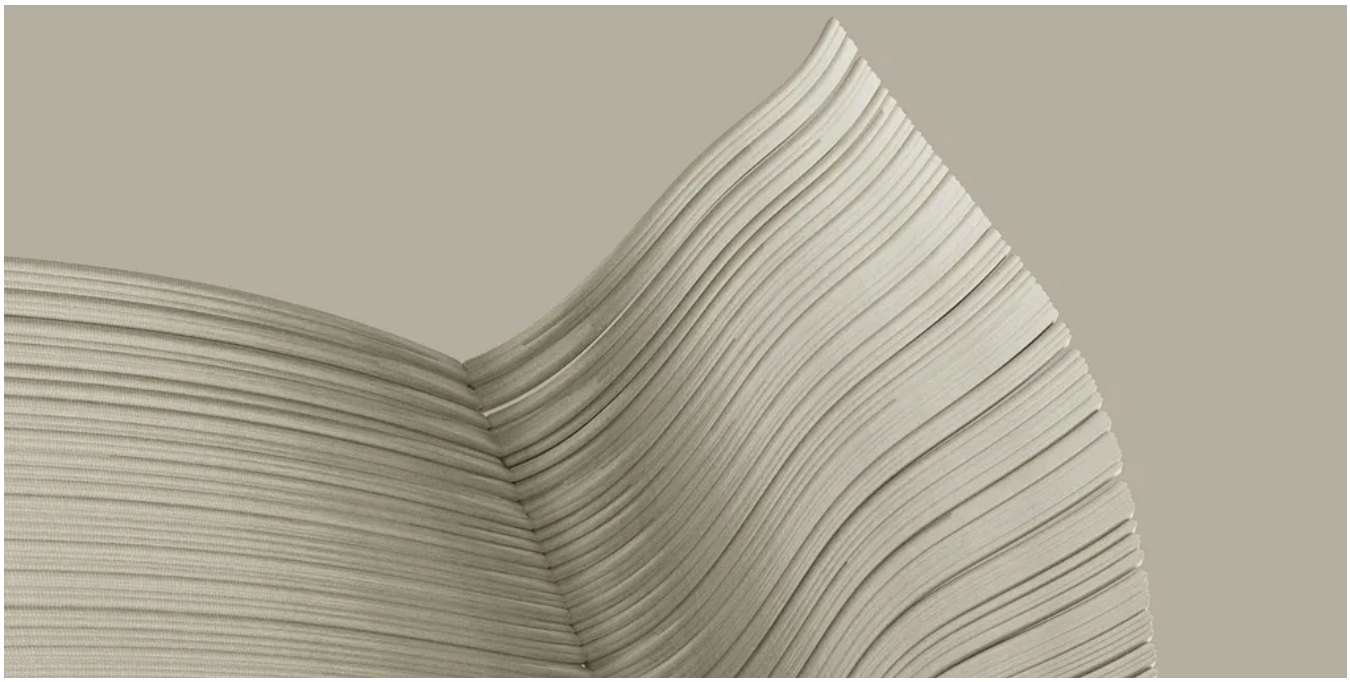
Learn to Deploy LLM Apps: Run Ollama on Google Colab and Connect Locally with Ngrok

Building an LLM project directly in a notebook can be challenging, especially when it comes to modularity and management. In this post...

May 25 57

See all from Ngoc

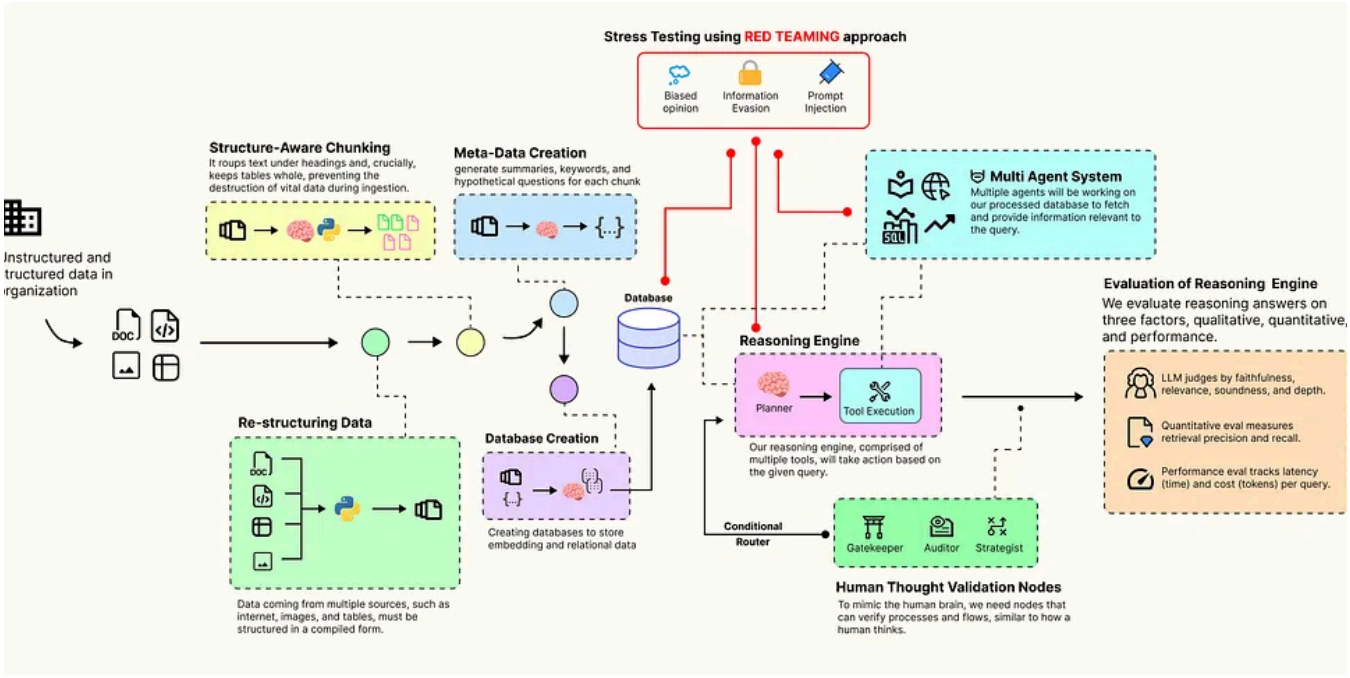
Recommended from Medium



In AI Advances by Alvis Ng

Chunking for LLMs: Windows, Retrieval, and Cost

4d ago 224

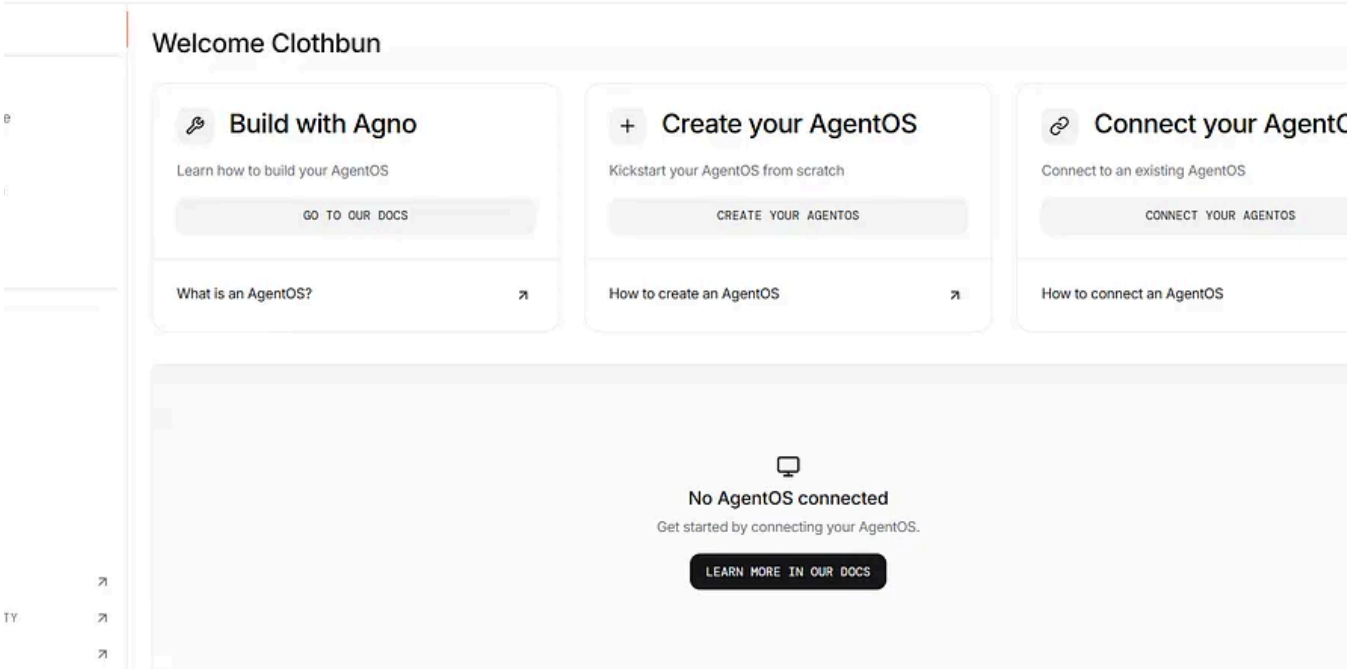


In Level Up Coding by Fareed Khan

Building an Advanced Agentic RAG Pipeline that Mimics a Human Thought Process

Ambiguity Checks, Multi-Tool Planning, Self-Correction, Causal Inference and more.

Sep 17 1.1K 16

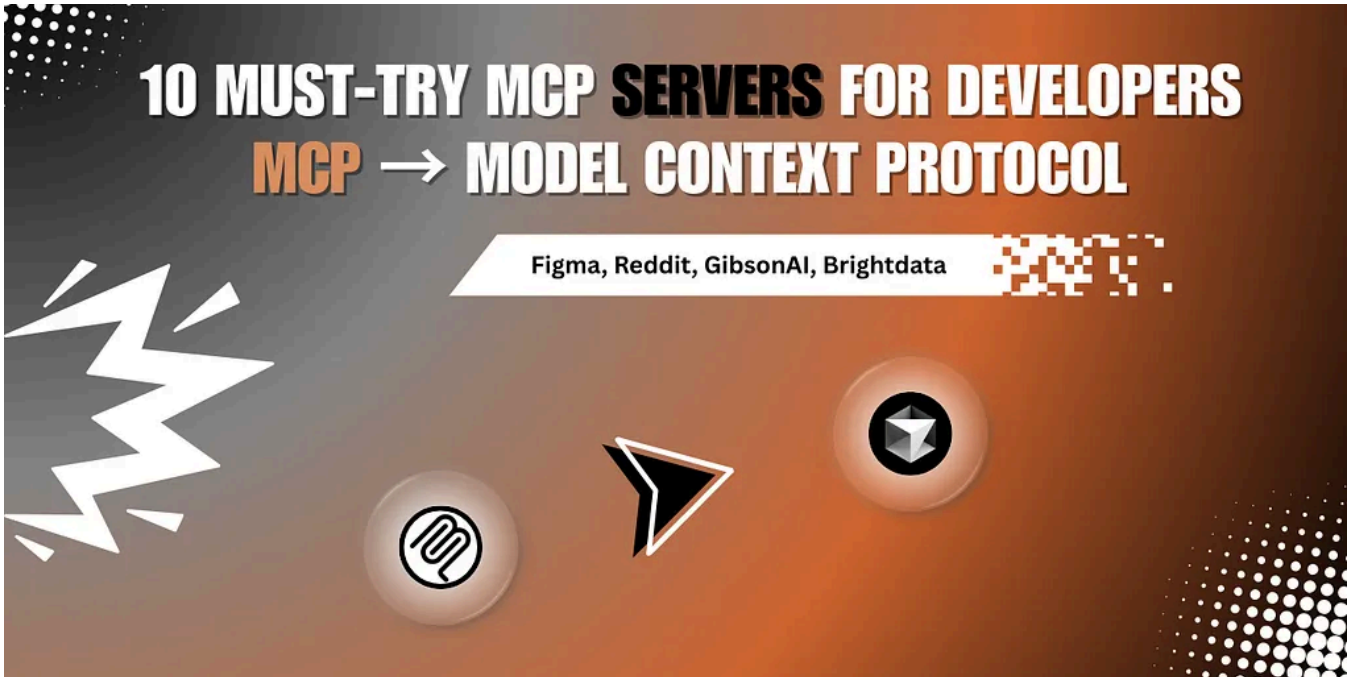


In Coding Nexus by Algo Insights

AgentOS: The New Operating System for AI Agents and How to Use It?

Every company already has its own operating system. Not the software kind, the complex mix of people, tools, docs, meetings, and processes...

Sep 22 65 1

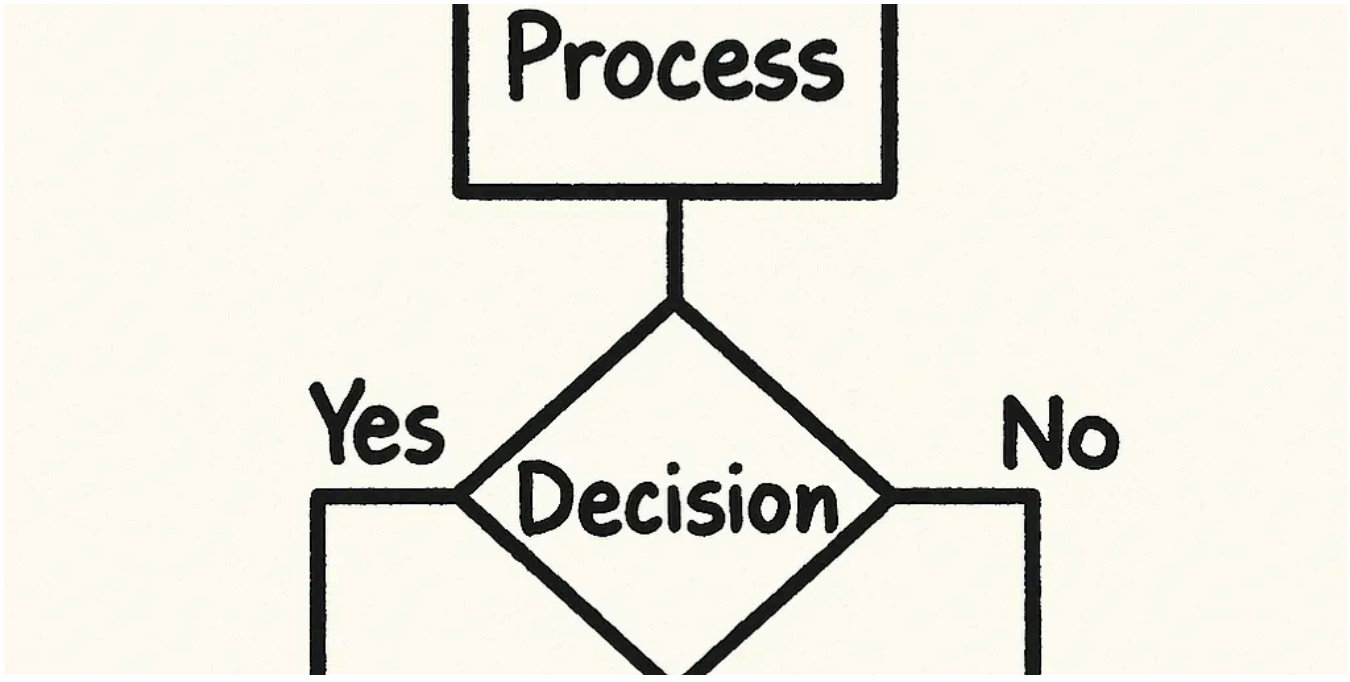


In Generative AI by Mr. Ånand

Model Context Protocol (MCP): 10 Must-Try MCP Servers for Developers

Explore Model Context Protocol (MCP)—a powerful way to connect tools, data, and models

Sep 15 575 6

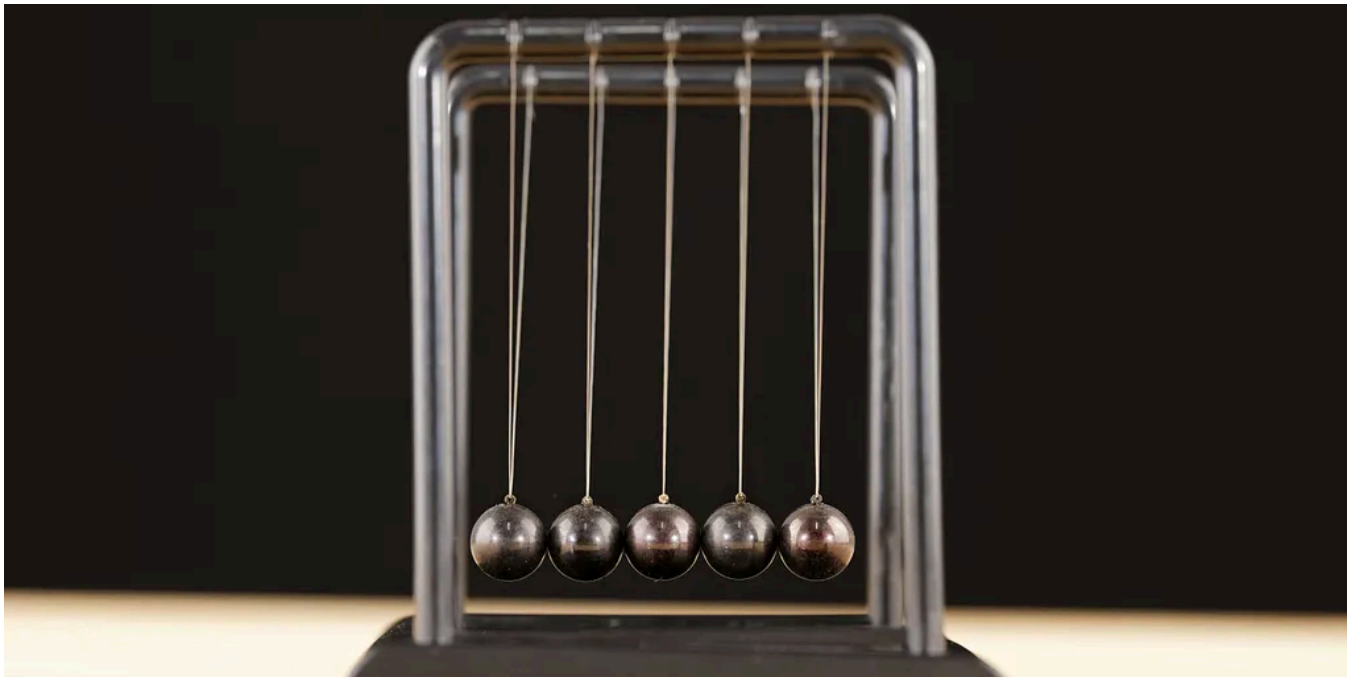


Berker Ceylan

What Is The Best Diagramming Software in 2025

I wasted three years diagramming in Lucidchart before a sketchy looking napkin app doubled my team's output.

Sep 11 725 24



Craig Adam

Agile is Out, Architecture is Back

The next generation of software developers will be architects, not coders.

6d ago 1K 44

See more recommendations